

Secret Breach Detection in Source Code with Large Language Models

Md Nafiu Rahman^{1,*} Sadif Ahmed^{1,*} Zahin Wahab^{2,*} S M Sohan³ Rifat Shahriyar¹

¹Bangladesh University of Engineering and Technology

²The University of British Columbia

³Google

*Equal contribution

ESEM 2025 | Technical Paper Track

The Problem: What Are Secrets in Code?

Secrets are the Keys to the Kingdom

Secrets are digital credentials (API keys, tokens, passwords) that grant access to sensitive systems and data.

- When hardcoded in source code, they become a major vulnerability.
- Pushing them to public repositories like GitHub leads to immediate exposure.

Example: A Hardcoded API Key (from Paper, Fig. 1)

```
const apiKey = "sk_live_...";  
const endpoint = "https://api.example.com/data";  
  
fetch(endpoint, {  
  method: "GET",  
  headers: {  
    "Authorization": `Bearer ${apiKey}`  
  }  
});
```

The Scale of the Problem: A Growing Epidemic

Secret Leaks are Exploding in Volume

This is a massive and rapidly growing threat across the software industry.

- **10 Million** hardcoded secrets were detected in 2022 alone, a **67% increase** from 2021 [4].
- Compromised credentials caused **half of all cyberattacks** in early 2023 (Sophos). [10]
- A report in 2024 highlighted a staggering **1212x surge** in leaked OpenAI API keys.[5]

The Limitations of Traditional Tools

Why Aren't Existing Tools Enough?

Most tools like Trufflehog [12], Gitrob [8], ggshield [3], and Gitleaks [6] rely on Regular Expressions (regex) [9] and entropy analysis.

- **The Flaw:** They lack **contextual understanding**.
- They can find patterns that *look like* secrets, but cannot distinguish between:
 - A real, active API key.
 - A placeholder string (e.g., 'API_KEY = "YOUR_KEY_HERE"').
 - A variable in a test file or an example in documentation.
- **The Consequence:** High false-positive rates lead to **"Alert Fatigue,"** causing developers to ignore warnings and miss real threats.

Our Solution: A Hybrid Approach with LLMs

Combining the Best of Both Worlds for High Accuracy

① Broad Candidate Extraction (Regex):

Use a comprehensive set of regex patterns to cast a wide net and identify all *potential* secrets. (Ensures High Recall)

Our Solution: A Hybrid Approach with LLMs

Combining the Best of Both Worlds for High Accuracy

① Broad Candidate Extraction (Regex):

Use a comprehensive set of regex patterns to cast a wide net and identify all *potential* secrets. (Ensures High Recall)

② Intelligent Classification (LLM):

Leverage the contextual reasoning of a Large Language Model to analyze the code around each candidate and filter out false positives. (Provides High Precision)

Goal: Drastically reduce false positives while maintaining high recall.

Related Work & Our Contribution

Building on Prior Work to Create a Better Solution

- **Secret Detection Tools (TruffleHog, Gitleaks):**
Effective regex/entropy scanners, but struggle with high false positives.
- **ML-Based Filtering (Saha et al.):**
Used traditional ML on handcrafted features; limited by lack of deep context.
- **Deep Learning (Feng et al.):**
Focused narrowly on textual passwords, not diverse secret types in code.

Our Novel Contribution

We are the first to systematically apply **fine-tuned, open-source LLMs** to provide deep contextual classification for a wide range of secrets directly in source code.

Research Questions

- **RQ1:** How effective are large, pre-trained language models (like GPT-4o) in detecting secrets without any fine-tuning?

Research Questions

- **RQ1:** How effective are large, pre-trained language models (like GPT-4o) in detecting secrets without any fine-tuning?
- **RQ2:** How effective are smaller, fine-tuned, open-source models (like LLaMA-3.1) for secure and efficient local deployment?

Methodology: The SecretBench Dataset

Foundation for Our Study

- We use **SecretBench** [1], a publicly available benchmark dataset.
- Contains **97,479 candidate secrets** extracted from 818 public GitHub repos.
- **15,084** of these are manually verified as **true secrets**.
- Spans **49 programming languages** and **311 file types**, ensuring real-world relevance.

Methodology: How We Question the LLMs

For each potential secret, we extract context and frame the problem as a classification task.

1. Context Extraction

- We take a **200-character window** of code surrounding the candidate string.

Methodology: How We Question the LLMs

For each potential secret, we extract context and frame the problem as a classification task.

1. Context Extraction

- We take a **200-character window** of code surrounding the candidate string.

2. Classification Tasks Posed to the LLM

- **Binary Classification:**
Is the candidate a Secret or Non-sensitive?

Methodology: How We Question the LLMs

For each potential secret, we extract context and frame the problem as a classification task.

1. Context Extraction

- We take a **200-character window** of code surrounding the candidate string.

2. Classification Tasks Posed to the LLM

- **Binary Classification:**

Is the candidate a Secret or Non-sensitive?

- **Multiclass Classification:**

If it is a secret, what kind is it? (e.g., API Key, Password, Access Token, etc.)

Methodology: Experimental Workflow

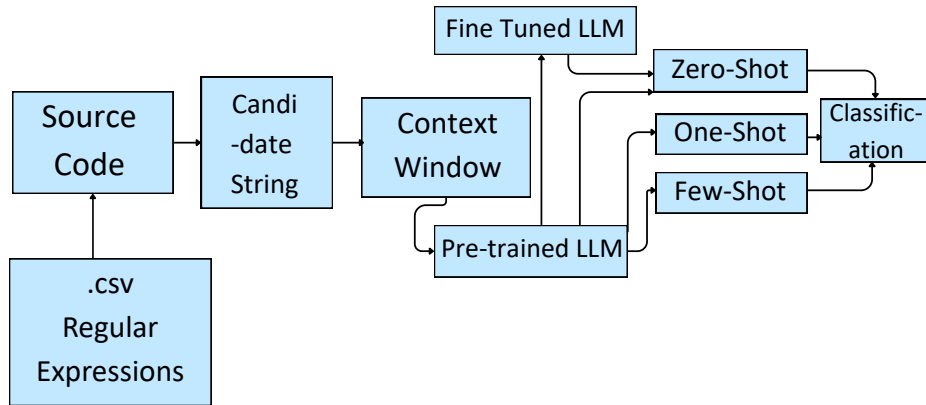


Figure: Our hybrid detection workflow.

RQ1 Results: Pre-trained LLMs (GPT-4o)

Powerful, But Not Perfect

GPT-4o performs well on binary classification with few-shot prompting but struggles with more nuanced multiclass classification. [1]

Table: GPT-4o Performance (Few-Shot)

Classification Task	F1-Score
Binary (Secret vs. Non-secret)	0.9392
Multiclass (Secret Type, weighted)	0.8108

Conclusion

Pre-trained models are strong for general detection but require fine-tuning for specialized, granular classification tasks.

RQ2 Results: The Power of Fine-Tuning

Fine-Tuning Unlocks State-of-the-Art Performance

We fine-tuned smaller, open-source models like LLaMA [11], Mistral [7] on the SecretBench dataset, resulting in a dramatic improvement and outperforming the much larger GPT-4o. [2, 3]

Table: Fine-Tuned Open-Source Model Performance

Classification Task	Top-Performing Model	F1-Score
Binary	LLaMA-3.1-8B	0.9852
Multiclass (weighted)	Mistral-7B	0.982

Key Takeaway

Fine-tuned smaller models are highly effective, secure, and efficient for local deployment.

Deep Dive: LLaMA-3.1-8B Binary Classification

A Near-Flawless Record (on a 3,000-sample test set)

- Correctly identified **1,463** of **1,500** secrets (True Positives).
- Correctly identified **1,496** of **1,500** non-secrets (True Negatives).
- Generated **only 4** false positives!

Deep Dive: Mistral-7B Multiclass Classification

Achieving Granular Understanding After Fine-Tuning

- The fine-tuned **Mistral-7B** model achieved a **weighted F1-Score of 0.982**.
- It excels at distinguishing between secret types, a task where pre-trained models struggled.

Discussion: What the Model Gets Right (True Positives)

The Model Understands Code Context

It correctly identifies secrets by recognizing common usage patterns.

Example 1: API Key

```
response = requests.get(  
    "https://api.stripe.com/v1/charges",  
    headers={"Authorization": f"Bearer API_KEY"}  
)
```

Example 2: Crypto Key

```
private_key = "-----BEGIN RSA..."  
decrypted = decrypt(data, private_key)
```

Discussion: What the Model Ignores (True Negatives)

Smarter Than Regex

The model correctly classifies non-sensitive strings that would fool a simple pattern matcher.

Example 1: Placeholder Key

```
# API_KEY = "xxxxxxxxxx"  
API_KEY = os.getenv("API_KEY")
```

Example 2: Test Credentials

```
def login_user(username, password):  
    if username == "user123":  
        print("Login successful")
```

Key Advantage

This contextual understanding is crucial for eliminating noise and developer alert fatigue.

Discussion: Why Did the Model Miss 37 Secrets?

Manual Analysis Revealed Three Key Causes

- ❶ **Labeling Inconsistencies:** Disagreements with the SecretBench dataset on ambiguous cases (e.g., public test keys).
- ❷ **Limited Input Size:** Long, multi-line secrets (like RSA keys) were truncated by the 200-character window.
- ❸ **Subtle Contextual Variations:** Rare cases where the model was sensitive to minor changes in the surrounding code.

Our Stance

We prioritized **high precision** (fewer false alarms) over catching every single edge case, which is the right trade-off for practical deployment.

Does This Work Outside of a Perfect Lab Setting?

- **Imbalanced Data:** We trained a model on a dataset that mimics real-world distribution (many non-secrets, few secrets). Performance barely dropped (1-3%), proving the models generalize well.

Discussion: Real-World Robustness

Does This Work Outside of a Perfect Lab Setting?

- **Imbalanced Data:** We trained a model on a dataset that mimics real-world distribution (many non-secrets, few secrets). Performance barely dropped (1-3%), proving the models generalize well.
- **Context Window Size:** Increasing the window from 200 to 300 characters offered only marginal gains.

Conclusion

The models are robust, and a 200-character window is highly effective and computationally efficient.

This is Practical and Deployable Today

- **Hardware:** All fine-tuning was done on a single consumer-grade **NVIDIA RTX 4090 GPU**.
- **Efficiency:** Using QLoRA [2], training VRAM usage was under 17GB.
- **Speed:** Inference takes just ~ 0.1 **seconds per candidate**, making it fast enough for real-time checks in CI/CD pipelines or IDEs.

Conclusion

- Fine-tuned LLMs offer a state-of-the-art solution for secret detection, achieving an **F1-score of 0.9852**.
- Smaller, open-source models are a practical, secure, and cost-effective alternative to commercial APIs for this task.
- Our hybrid approach significantly reduces the false positives that plague traditional tools.

Future Work

- Integrate into CI/CD pipelines and IDEs.
- Explore ensemble models to further reduce false negatives.
- Extend detection to other artifacts (e.g., config files, documentation).

References I

-  Setu Kumar Basak, Lorenzo Neil, Bradley Reaves, and Laurie Williams. Secretbench: A dataset of software secrets. *arXiv preprint arXiv:2303.06729*, 2023.
-  Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms, 2023.
-  GitGuardian. ggshield. <https://www.gitguardian.com/ggshield>, 2017. Accessed: 2024-02-02.
-  GitGuardian. State of secrets sprawl report 2023, 2024.

References II



GitGuardian.

The state of secrets sprawl report 2024.

<https://www.gitguardian.com/state-of-secrets-sprawl-report-2024>, 2024.

Accessed: 2025-04-16.



Gitleaks.

Gitleaks.

<https://github.com/gitleaks/gitleaks>, 2018.

Accessed: 2024-02-02.



Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, L  lio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timoth  e Lacroix, and William El Sayed.

Mistral 7b, 2023.

References III



Michenriksen.
gitrob, 2014.



Truffle Security.
Regular expressions used in trufflehog.
<https://github.com/trufflesecurity/trufflehog/tree/main/pkg/detectors>,
2016.
Accessed: 2024-03-27.



Sophos.
Sophos 2023 threat report.
<https://www.sophos.com/en-us/content/security-threat-report>, 2023.
Accessed: 2024-03-21.

References IV

-  Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample.
Llama: Open and efficient foundation language models, 2023.
-  TruffleSecurity.
Trufflehog.
<https://github.com/trufflesecurity/trufflehog>, 2016.
Accessed: 2024-02-02.

Thank You

Replication Package

Our code and data are available here [link]

Contact or Questions?

nafiu.rahman@gmail.com